

Reviewer Pack — Noncommutative Fourier Coefficient Count Lower Bound for AC^0 ...

Entry 1c23316cce55 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 22:59:55 UTC

Noncommutative Fourier Coefficient Count Lower Bound for AC^0 PARITY Circuits

Entry ID: 1c23316cce55 Verdict: INCONCLUSIVE Recorded: 2026-05-19 22:59:55 UTC

1. Conjecture

Field A (mathematical branch): Noncommutative Harmonic Analysis **Field B** (complexity object): AC^0 Circuit Size for PARITY

Statement:

For any AC^0 circuit C of depth d computing the PARITY function on n variables, the number of non-zero coefficients in the noncommutative Fourier transform of PARITY is $\Omega(n^{\{1/(d-1)\}})$.

Rationale (proposer's reasoning):

The noncommutative Fourier transform captures the algebraic structure of PARITY in a noncommutative setting. Its coefficient count reflects the depth required to compute PARITY, aligning with the switching lemma's lower bounds on AC^0 circuit size.

Taxonomy category: NISAN_WIGDERSON_DESIGNS (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 712232062c8c974e

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	SAFE	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def noncommutative_fourier_transform(n):
        if n <= 0:
            return []
        transform = [0] * (1 << n)
        for i in range(1 << n):
            sign = (-1) ** bin(i).count('1')
            transform[i] = sign
        return transform

    def count_nonzero_coefficients(transform):
        return sum(1 for coeff in transform if coeff != 0)

    def lower_bound(n, d):
        if d <= 1:
            return float('-inf')
        return n ** (1 / (d - 1))

    n = random.randint(5, 40)
    d = random.randint(2, 3) # AC0 depth is typically small
    transform = noncommutative_fourier_transform(n)
    nonzero_count = count_nonzero_coefficients(transform)
    expected_lower_bound = lower_bound(n, d)

    return {
        "metric_name": "nonzero_coefficient_count",
        "metric_value": nonzero_count,
        "instances_tested": 1,
        "conjecture_holds": nonzero_count >= expected_lower_bound,
        "counterexample": "" if nonzero_count >= expected_lower_bound else f"n={n}, d={d}"
    }

if __name__ == "__main__":
    import sys
```

```

seeds = [int(s) for s in sys.argv[1:]] or list(range(2, 30*31, 2)) # Default to first 30 primes

results = []
for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

total_nonzero_count = sum(r["metric_value"] for r in results)
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

mean = Fraction(total_nonzero_count, len(results))
std_dev = math.sqrt(sum((r["metric_value"] - mean) ** 2 for r in results) / len(results))

if support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean} std={std_dev} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample={r['counterexample']}\n first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE budget_exceeded n_tested=30")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6602b959.py", line 58, in <module>
    result = run_trial(seed)
              ~~~~~^~~~~~

  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6602b959.py", line 40, in run_trial
    transform = noncommutative_fourier_transform(n)
                  ~~~~~^~~~~~

  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6602b959.py", line 24, in noncommutative_fourier_transform
    transform = [0] * (1 << n)
                  ~~~~~^~~~~~

MemoryError

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with MemoryError before producing results, preventing evaluation of conjecture. | next: Optimize noncommutative Fourier transform implementation to handle larger n values

11. Audit log (LLM calls)

Total LLM calls: 12

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	136367
2	propose	ollama_remote	qwen3:8b	0	0	203587
3	propose	ollama_remote	qwen3:8b	0	0	143004
4	propose	ollama_remote	qwen3:8b	0	0	130690
5	preregistration	ollama_remote	qwen3:8b	0	0	31675
6	novelty	ollama_remote	qwen3:8b	0	0	27482
7	novelty	ollama_remote	qwen3:8b	0	0	17554
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12122
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7635
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6635
11	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7165
12	judge	ollama_remote	qwen3:8b	0	0	85691

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 809606 ms total latency. Provider mix: {'ollama_remote': 12}

(full prompt+response transcripts available in *research/audit/1c23316cce55.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/1c23316cce55.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/1c23316cce55.tar.gz* (if generated)